

Devoir surveillé terminal

Durée 1h30
(Corrections)

Exercice 1 (File de messages partagée (les IPC) - 10 pts)

Nous souhaitons mettre en place une architecture distribuée de communication entre des clients et un contrôleur à l'aide d'une file de messages partagée. Les clients peuvent déposer des messages qui sont lus par les autres clients ou le contrôleur. Seul le contrôleur peut supprimer les messages de la file.

Chaque client est caractérisé par son statut : un PID, la date de connexion et la date de dernière émission dans la file. Le nombre maximum de clients est fixé à l'aide d'une constante $MAX_CLT = 10$. La file de messages, représentée par un tableau, contient un nombre maximal d'entrées fixé par une constante $MAX_ENT = 20$. Chaque entrée est caractérisée par le PID de l'émetteur et un message (une chaîne de caractères de taille fixe spécifiée par la constante $MAX_MSG = 256$). La file de messages et les statuts des clients sont stockés dans un segment de mémoire partagée. La gestion de la concurrence est réalisée à l'aide de sémaphores.

Pour accéder à la file, chaque client doit être connecté : il doit d'abord trouver une place disponible parmi les statuts et spécifier son statut.

Pour rappel, une file circulaire de taille T représentée à l'aide d'un tableau utilise deux entiers : les indices de début (D) et de fin (F). Lorsque $F=D$, la file est vide. Lorsque $(D+1)\%T=F$, la file est pleine (en réalité, la file ne peut contenir que $T-1$ éléments). Le prochain élément à lire est situé dans la case dont l'indice est F (ensuite $F=(F+1)\%T$). Un nouvel élément doit être placé dans la case d'indice D (ensuite $D=(D+1)\%T$).

1°) Expliquez pourquoi une file de messages IPC ne serait pas appropriée dans le cadre de cette application. (1 point)

Solution : La lecture dans une file supprime les messages. Du coup, si un client souhaite lire les messages des autres clients, ceux-ci sont supprimés et il faut ensuite les replacer. Cela serait trop coûteux en termes d'appels systèmes et changerait l'ordre des messages.

2°) Donnez le contenu du segment de mémoire partagée, en précisant le type des données. Spécifiez les structures nécessaires (statut et message). (2 points)

Solution : Le statut est caractérisé par la structure suivante :

```
typedef struct {
    pid_t pid; /* PID du processus */
    time_t connexion; /* Date de connexion */
    time_t emission; /* Dernière émission */
} statut_t;
```

Un message de la file est caractérisé par la structure suivante :

```
typedef struct {
    pid_t pid; /* PID du processus */
    char message[MAX_MSG]; /* Le message */
} message_t;
```

Le segment est donc constitué des éléments suivants :

- Les statuts : `statut_t[MAX_CLT]`
- La file de messages :

- deux entiers (F et D)
- les entrées : `message_t[MAX_ENT+1]`

Le tableau de sémaphores et le segment de mémoire partagée sont créés et initialisés par le contrôleur. Nous supposons que la limitation du nombre de clients est gérée à l'aide d'un sémaphore. Lorsqu'un client démarre, si le nombre maximal de clients est atteint, il doit s'arrêter et ne pas resté bloqué. Dès qu'un client se connecte, il spécifie son statut dans le segment de mémoire partagée dans un emplacement libre. Il peut ensuite ajouter des messages dans la file ou les consulter.

3°) Donnez les sémaphores nécessaires pour la phase de connexion, ainsi que leur initialisation. (1 point)

Solution : Nous avons besoin d'un sémaphore pour le nombre de clients (S_1) initialisé à MAX_CLT et d'un sémaphore pour permettre l'exclusion mutuelle sur les statuts (S_2) initialisé à 1.

4°) Donnez toutes les étapes nécessaires du démarrage d'un client jusqu'à la spécification de son statut dans le segment de mémoire partagée. Vous préciserez tous les appels systèmes ainsi que les sémaphores nécessaires (en indiquant leur valeur d'initialisation). (3 points)

Solution : Le client doit récupérer le tableau de sémaphores (`semget`). Il doit ensuite faire une opération non bloquante pour décrémenter le sémaphore S_1 (`semop`). Si l'opération échoue, le client s'arrête. Sinon, il récupère le segment de mémoire partagée (`shmat`) et l'attache à sa mémoire (`shmat`). Il doit demander l'exclusion mutuelle sur les statuts avec le sémaphore S_2 (`semop`) puis rechercher dans le tableau un statut libre ($PID=0$, par exemple). Il spécifie ensuite son statut puis libère l'exclusion mutuelle sur le sémaphore S_2 (`semop`).

Pour éviter que des messages ne soient perdus, il faut empêcher les clients d'envoyer un message si la file est pleine. Par contre, les autres clients et le contrôleur doit pouvoir continuer à lire les messages.

5°) Proposez une solution à l'aide de sémaphores, tout en limitant le nombre d'appels systèmes et en évitant l'attente active. Décrivez précisément le fonctionnement de votre solution (ajout et lecture des messages). (3 points)

Solution : Une solution consiste à utiliser un sémaphore S_3 initialisé à MAX_ENT et un second S_4 initialisé à 1. Lorsqu'un client désire écrire un message, il doit décrémenter de 1 les deux sémaphores en réalisant les deux opérations en un seul appel atomique. Si la file est pleine, il sera bloqué avec le sémaphore S_3 . Si le contrôleur ou un autre client est en train de lire ou d'écrire dans la file, il sera également bloqué avec le sémaphore S_4 . Ainsi, il ne peut écrire que si les deux opérations sont possibles. Une fois le message ajouté, il peut incrémenter S_4 de 1 pour libérer l'exclusion mutuelle. Le contrôleur ou un client qui désire lire les messages doivent simplement décrémenter le sémaphore S_4 . Si un client est en train de lire ou d'écrire, ils seront bloqués. Pour chaque message supprimé, le contrôleur incrémente S_3 . Une fois que l'accès à la file n'est plus nécessaire, on incrémente S_4 pour libérer l'exclusion mutuelle.

Exercice 2 (Jeu en réseau (les sockets) - 10 pts)

Nous souhaitons développer un jeu multi-joueurs en réseau. Nous supposons deux applications : un client (exécuté pour chaque joueur) et un serveur. Le serveur se met en attente de requêtes sur un numéro de port UDP donné. Il peut recevoir deux types de requêtes : une demande de création d'une partie (en spécifiant le nombre de joueurs attendus) et une demande de la liste des parties créées.

Lorsqu'une demande de création de partie est reçue, le serveur crée une socket TCP qui permettra de mettre en relation les joueurs de cette partie. Le choix du numéro de port est laissé à la charge du système d'exploitation et non par l'application. L'adresse est envoyée au client qui a demandé la création de la partie pour qu'il puisse s'y connecter. La socket est gérée ensuite par un fils du serveur : gestion des connexions des clients et déroulement de la partie (une fois tous les clients connectés).

Dans un premier temps, une partie est simplement caractérisée par l'adresse de la socket TCP (adresse et numéro de port).

Nous ne considérons pas les pannes de type déconnexion dans cet exercice.

1°) Expliquez toutes les actions réalisées par le serveur à partir de son démarrage jusqu'à l'attente de requêtes sur la socket UDP. Précisez les appels systèmes utilisés. (1 point)

Solution : *Le serveur doit créer une socket de type UDP avec l'appel `socket` et la nommer avec l'appel `bind`. Il attend ensuite les requêtes avec `recvfrom`.*

2°) Expliquez les actions du serveur lorsqu'il reçoit une demande de nouvelle partie, jusqu'à la réponse au client et la mise en place de son fils. Précisez les appels systèmes. (2 points)

Solution : *Lorsque le serveur reçoit la requête du client avec `recvfrom`, il récupère l'adresse du client. Il crée une nouvelle socket TCP (`socket`), la nomme (`bind`) avec un numéro de port 0 (le système choisit lui-même un port aléatoire). L'adresse (donc le numéro de port) est récupérée à l'aide de `getsockname`. Le serveur crée un fils avec `fork` et lui passe le descripteur de la socket TCP créée. Il répond ensuite au client en utilisant l'adresse récupérée par `recvfrom` et lui envoie l'adresse de la socket TCP. Cette adresse peut être conservée dans une liste pour avoir la liste des parties.*

3°) Décrivez toutes les actions du client souhaitant créer une partie jusqu'à l'accès à cette partie. Précisez les appels systèmes. (2 points)

Solution : *Le client crée d'abord une socket avec l'appel `socket`. Il crée ensuite l'adresse du serveur qu'il spécifie à `sendto`. Il se met ensuite en attente d'une réponse du serveur avec `recvfrom`. Il crée une nouvelle socket TCP avec `socket` et se connecte au fils du serveur à l'aide de `connect` avec l'adresse reçue dans la réponse. La partie peut ensuite commencer.*

4°) Comment le serveur peut-il conserver une trace des parties pour les fournir à la demande? Comment peut-il savoir quand une partie se termine? Précisez les appels systèmes éventuels nécessaires. Vous ne devez pas créer de nouvelles ressources systèmes (fichier, tube, etc.). (2 points)

Solution : *Quand la socket est créée, la méthode `getsockname` permet de récupérer l'adresse de la socket et donc le numéro de port attribué par le système. Cette adresse est placée dans une liste. Comme la socket est ensuite gérée par le fils du serveur, il faut que le fils avertisse le père lors de la fin d'une partie. Par exemple, il peut envoyer une requête sur le port UDP du serveur (`sendto`). Une autre solution consiste à attendre la fin du fils sur le serveur. Lors de la création des fils, le serveur conserve leur PID qu'il associe aux parties créées. Il met ensuite en place un gestionnaire sur le signal `SIGCHLD` à l'aide de `sigaction`. Lorsqu'il reçoit un signal `SIGCHLD`, le serveur sait que le fils est terminé (et donc la partie). Il peut récupérer le PID du fils avec `wait`. Il supprime alors la partie correspondante.*

5°) Nous souhaitons maintenant que le serveur conserve le nombre de clients actuellement connectés dans chaque partie pour l'indiquer aux clients qui souhaitent obtenir la liste des parties. Proposez une solution en précisant les appels systèmes éventuels nécessaires. Vous ne devez pas créer de nouvelles ressources systèmes (fichier, tube, etc.). (1 point)

Solution : *Lorsqu'un client se connecte, le fils peut envoyer une requête sur la socket UDP du serveur. Le serveur se contente alors d'incrémenter de 1 le nombre de clients à la réception de ce type de requête.*

Nous supposons que pour chaque client qui se connecte, le fils du serveur crée un fils. Nous supposons également que les clients communiquent en s'échangeant des messages représentés par la structure de type `message_t`. Les clients peuvent envoyer des messages soit à un client en particulier, soit à tous les autres clients.

6°) Expliquez comment mettre en place les communications (la transmission des messages entre les clients). Précisez les appels systèmes nécessaires. (2 points)

Solution : *Si les fils sont créés à chaque connexion de client, ils n'auront pas accès aux sockets de communication des autres clients. Une solution est de créer un tableau qui contiendra toutes les sockets de communication puis de créer les fils dédiés à la gestion des clients ensuite, une fois la partie démarrée. Lorsqu'un message est reçu avec `recvfrom`, le fils écrit directement le message dans la socket correspondante au destinataire*

avec **sendto**. Par contre, il faut gérer la concurrence. Nous pouvons utiliser un sémaphore initialisé à 1 pour chaque socket ou bien un sémaphore pour l'ensemble des sockets. Il faut utiliser **semget** pour récupérer le tableau de sémaphores, et **semop** pour les opérations.